

Linux Shell

Check system kernel and hardware:

Terminal:

`uname // Ex: linux`

`uname -a // show all information`

`lsb_release -i // Ex: ubuntu`

`lsb_release -d // Ex: ubuntu 24.04.2 LTS`

`lsb_release -id // the two functions together`

Check CPU information:

Terminal:

`cat /proc/cpuinfo`

Check memory information:

Terminal:

`cat /proc/meminfo`

Show text with numeric lines:

Terminal:

`nl example.txt`

Show & Edit & create text/file:

Terminal:

`nano example.txt // Dans la terminal`

`gedit example.txt // Dans un editeur`

`touch example.anything // create a file`

`cat example.txt // show the text in the terminal`

`tac example.txt // show the text but in reverse from the last line to the first`

Show first lines of a file:

Terminal:

`head example.txt //show first 10 lines of the document.`

`head -x example.txt //show first x lines of the document.`

Show last lines & Show from line x of a file:

Terminal:

`tail example.txt //show the last 10 lines of a document.`

`tail -x example.txt //show the last x lines of a document.`

`tail +x example.txt //show from line x the document.`

Search something in a file and show the line of it:

Terminal:

`grep "object" file.txt`

`grep -r "object" directory`

`grep -r "object" *`

Create a directory_s:

Terminal:

`mkdir DirectoryName // to create one directry in the corrent location`

`mkdir -p dir1/dir2/dir3 // to create the full path of the directorys`

`mkdir -p dir1/dir2/{sdir1,sdir2}/dir3/{sdir1,sdir2} // to create the full tree of folders you want`

Create a program shell:

Terminal:

```
//create the file an access it using:
nano example.sh
//inside nano you should write this:
#!/bin/bash
// now write any code you want to execute
//save the file
// after this you can sheck if your file is executable or not using:
ls -l example.sh
//if it is not executable write this to make it so:
chmod +x example.sh
//you can now execute your file using:
bash example.sh
or:
./example.sh
```

The && Operator:

You can write two operation in the same line and each one will operate only if the one before it is executed successfully.

For example:

Terminal:

```
mkdir test && cd test
```

The | Operator:

| (Pipe): Takes the standard output of the command on the left and uses it as the standard input for the command on the right.

For example:

Terminal:

```
ls -l | less
```

The > Operator:

> (Redirect Output): Overwrites a file with the standard output of a command.

For example:

Terminal:

```
ls -l > file_list.txt
```

The >> Operator:

>> (Append Output): Appends the standard output of a command to the **end** of a file.

Softlink:

Create a link like in windows

Terminal:

```
ln -sr <file/folder you want to make a shortcut for>  
<NewName/destination for the shortcut>
```

Create New Group:

Terminal:

```
groupadd groupname
```

Change Group file/directory owner:

Terminal:

```
chgrp groupname <file/folder>
```

Create New user:

Without directory:

Terminal:

```
useradd username
```

With directory:

Terminal:

```
useradd -m username //This make a directory for it in home
```

```
useradd -m -d /path/folder_name username // This make the  
directory in other path with the name you want
```

First Group, Second Group:

you can add -g groupname for the first group and -G group name for the second group

Create new password for your user:

Terminal:

```
sudo passwd username
```

Edit user:

Terminal:

```
usermod -g groupname -G groupname username
```

Change user in the terminal

Terminal:

```
sudo su – username
```

// better to check if the user is running on shell or bash, if in shell you can change it by `usermod -s /bin/bash username`

Delete user:

Terminal:

```
userdel username // to delete the user and keep his folders
```

```
userdel -r username // to delete the user and delete his folders
```

Change user/group file/directory owner

Terminal:

```
chown username <file/folder>
```

// !Also you can Add -R to make it recursive with a folder

`chown username:groupname <file/folder> // to change the user and group at the same time`

`chown :groupname <file/folder> // to change the group`

Bash program shell (variable, list, condition, loops):

Variables:

Terminal:

`variable = value // simple`

`read -p "give the value of the variable: " variable // this is an input`

List:

Terminal:

`tableau=( "element1" "element2" "elementN" )`

`echo "${tableau[*]}"`

Condition:

Terminal:

`rep = <path>`

`if [ -d "$rep" ]; then`

`echo "$rep is a directory"`

`else`

`echo "$rep is a file"`

`fi`

Terminal:

`if [ "$number1" -ge "number2" ]; then`

`echo "$number1 is greater or equal to $number2"`

`fi`

Loops:

Terminal:

`for I in {1..5}; do`

`echo "$I"`

done

Changing the access of a file/folder:

Terminal:

`chmod +x // add access permission to username`

`chmod u+x // same`

`chmod g+x // for group`

`chmod o+x // for other`

`chmod ugo+x // for all of them`

`chmod u=rwx, g=r,o=wx //simple each one with his access`

`chmod 743 // same but with numbers in octale`

Other commands:

Terminal:

`rm // remove a file`

`rmdir // remove an empty directory`

`rm -r // remove directory with the inside of it`

`mv // change the location of something or rename something`

`cp // to copy files`

`cp -r // to copy directory`

`ls -a // list all even hidden ones`

`ls -d // list directory not their content`

`ls -ad [a-c]* // list all hidden files and directory that start with a,b,c and not the directory content`

`ls -ad [a,f]* // list all hidden files and directory that start with a or f and not the directory content`

`pwd // path of where you are`

`who // who is currently logged into the system`

